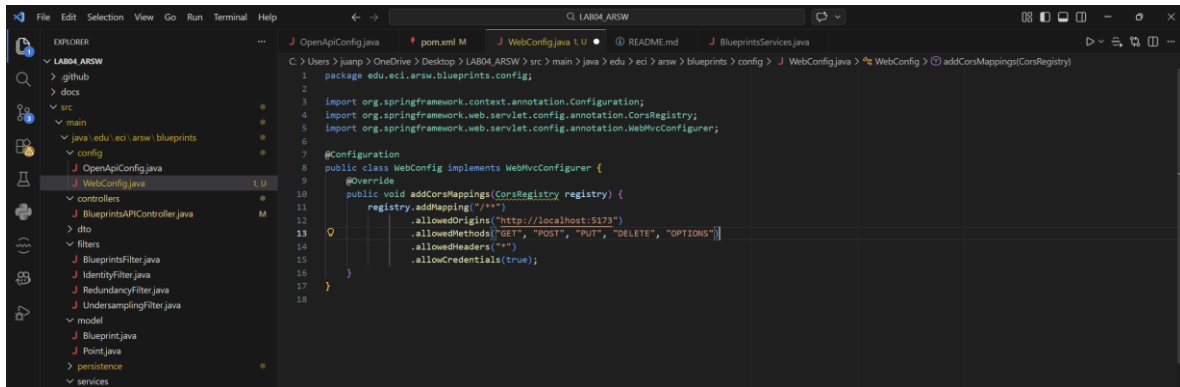


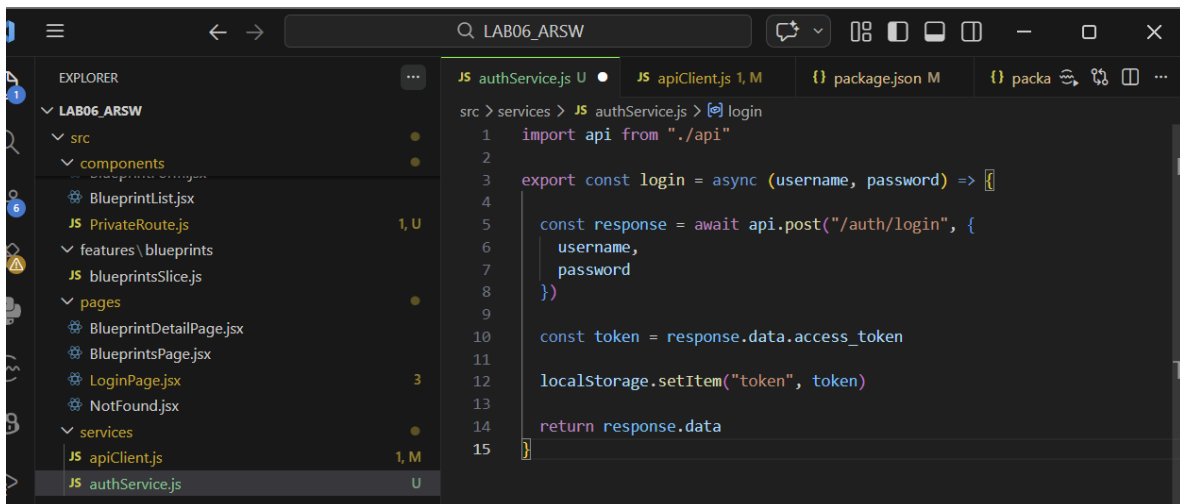
- Se añadió configuración de CORS para controlar qué páginas web pueden hacer peticiones al backend.



The screenshot shows an IDE with a project named LAB04_ARSW. The file explorer on the left shows a directory structure with files like pom.xml, WebConfig.java, and various controllers and filters. The main editor window displays the content of WebConfig.java, which implements WebMvcConfigurer. The addCorsMappings method is overridden to allow requests from http://localhost:5173 with GET, POST, PUT, DELETE, and OPTIONS methods, and all headers. Credentials are also allowed.

```
1 package edu.eci.arsw.blueprints.config;
2
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.web.servlet.config.annotation.CorsRegistry;
5 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
6
7 @Configuration
8 public class WebConfig implements WebMvcConfigurer {
9     @Override
10     public void addCorsMappings(CorsRegistry registry) {
11         registry.addMapping("/**")
12             .allowedOrigins("http://localhost:5173")
13             .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
14             .allowedHeaders("*")
15             .allowCredentials(true);
16     }
17 }
18
```

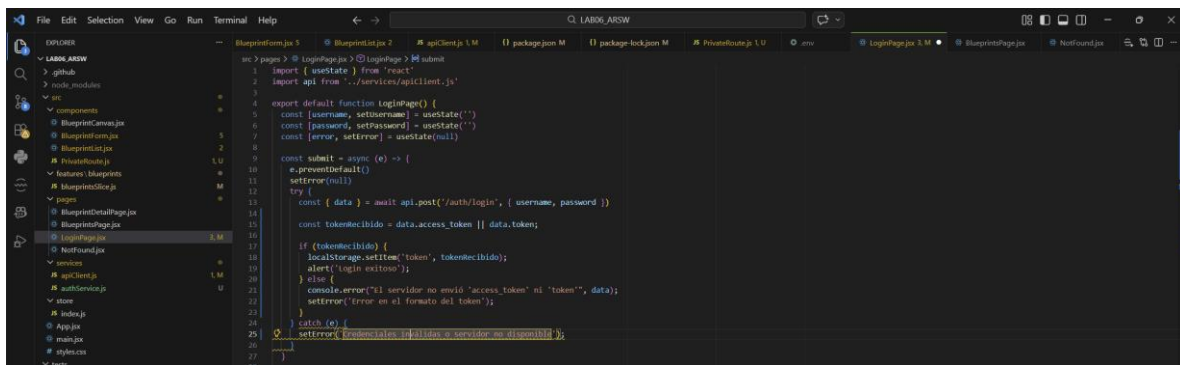
- Permitir validar el token y loguearse



The screenshot shows an IDE with a project named LAB06_ARSW. The file explorer on the left shows a directory structure with files like PrivateRoute.js, blueprintsSlice.js, LoginPage.jsx, and authService.js. The main editor window displays the content of authService.js, which defines a login function. The function uses an API client to post data to /auth/login and returns the access token from the response data.

```
1 import api from "/api"
2
3 export const login = async (username, password) => {
4
5     const response = await api.post("/auth/login", {
6         username,
7         password
8     })
9
10    const token = response.data.access_token
11
12    localStorage.setItem("token", token)
13
14    return response.data
15 }

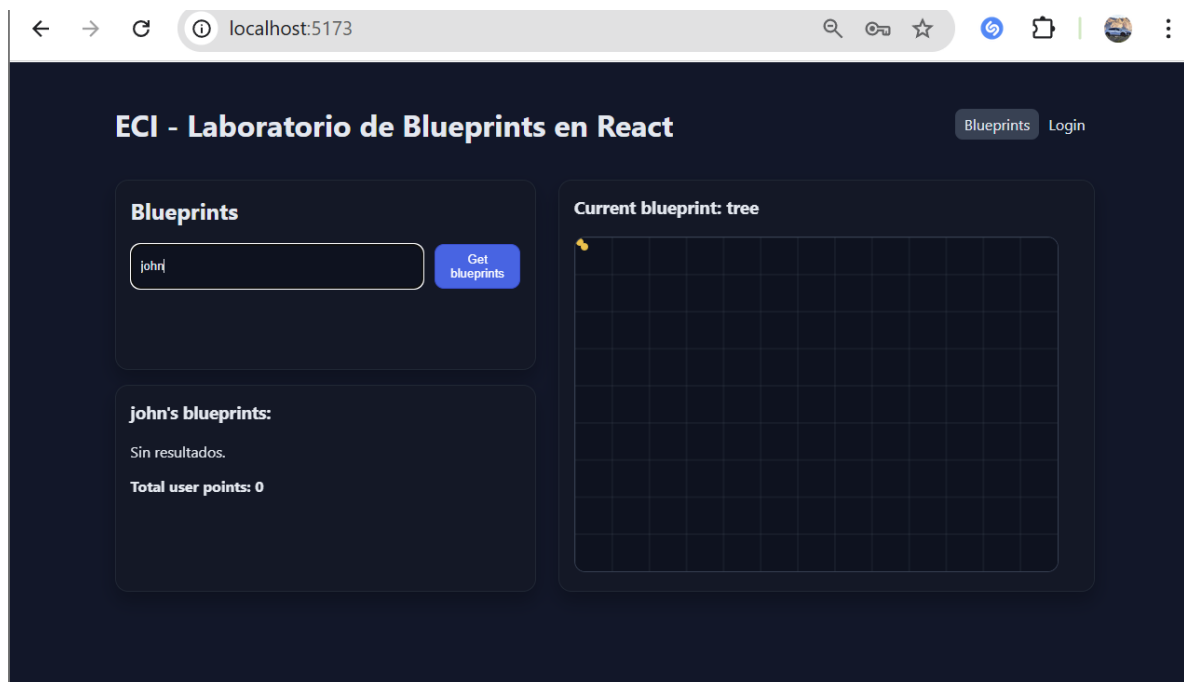
```



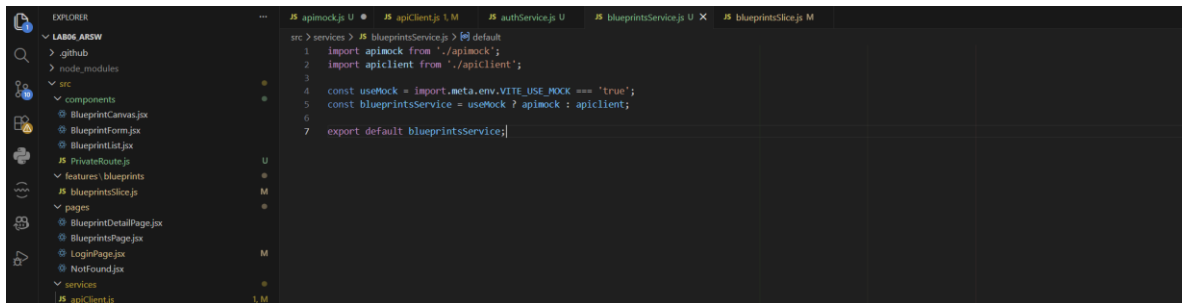
The screenshot shows an IDE with a project named LAB06_ARSW. The file explorer on the left shows a directory structure with files like LoginPage.jsx, authService.js, and apiClient.js. The main editor window displays the content of LoginPage.jsx, which is a functional component. It uses useState to manage username, password, and error state. It has a submit function that calls the login function from authService.js and updates the error state if there is an error.

```
1 import { useState } from 'react'
2 import api from "/api"
3
4 export default function LoginPage() {
5     const [username, setUsername] = useState("")
6     const [password, setPassword] = useState("")
7     const [error, setError] = useState(null)
8
9     const submit = async (e) => {
10         e.preventDefault()
11         setError(null)
12         try {
13             const data = await api.post("/auth/login", { username, password })
14             const tokenRecibido = data.access_token || data.token;
15
16             if (tokenRecibido) {
17                 localStorage.setItem('token', tokenRecibido);
18                 alert('login exitoso');
19             } else {
20                 console.error("El servidor no envió 'access_token' ni 'token'");
21                 setError("Error en el formato del token");
22             }
23         } catch (e) {
24             setError("Problemas al intentar conectarse al servidor no disponible");
25         }
26     }
27 }

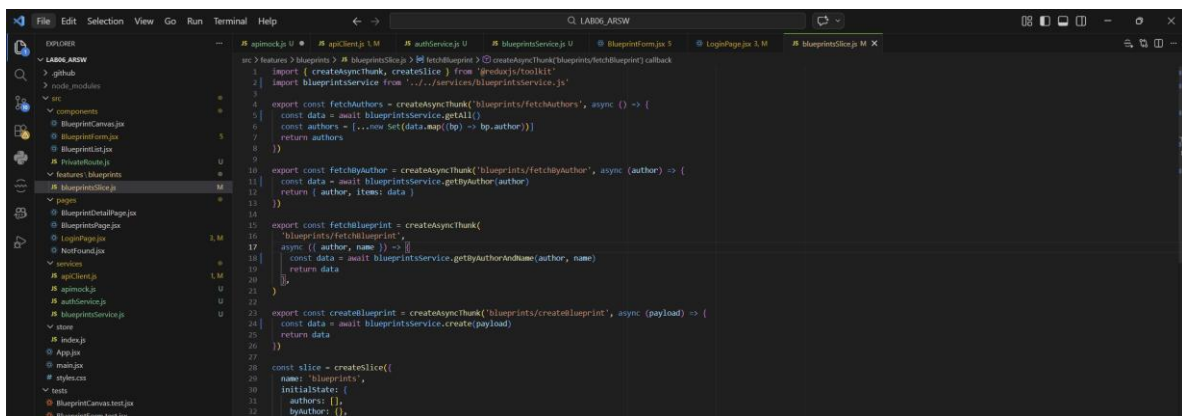
```



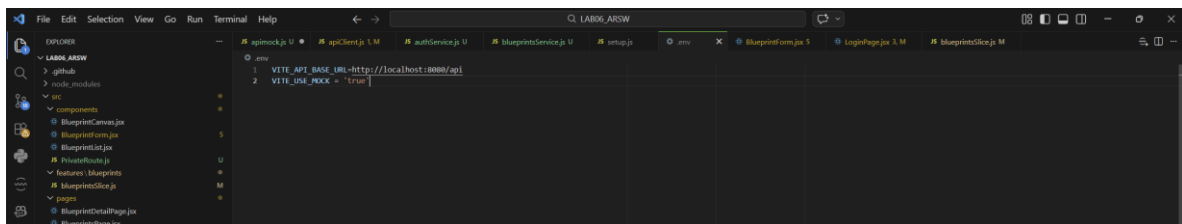
- Se implementó un sistema de servicios conmutables que permite alternar entre una fuente de datos simulada (**apimock**) y el servidor real (**apiclient**) mediante la variable de entorno VITE_USE MOCK. Esta arquitectura de servicios desacoplados garantiza que la lógica de Redux sea agnóstica a la procedencia de los datos, facilitando el desarrollo independiente del frontend y asegurando la consistencia de la interfaz a través de métodos estandarizados como getByAuthor y create.



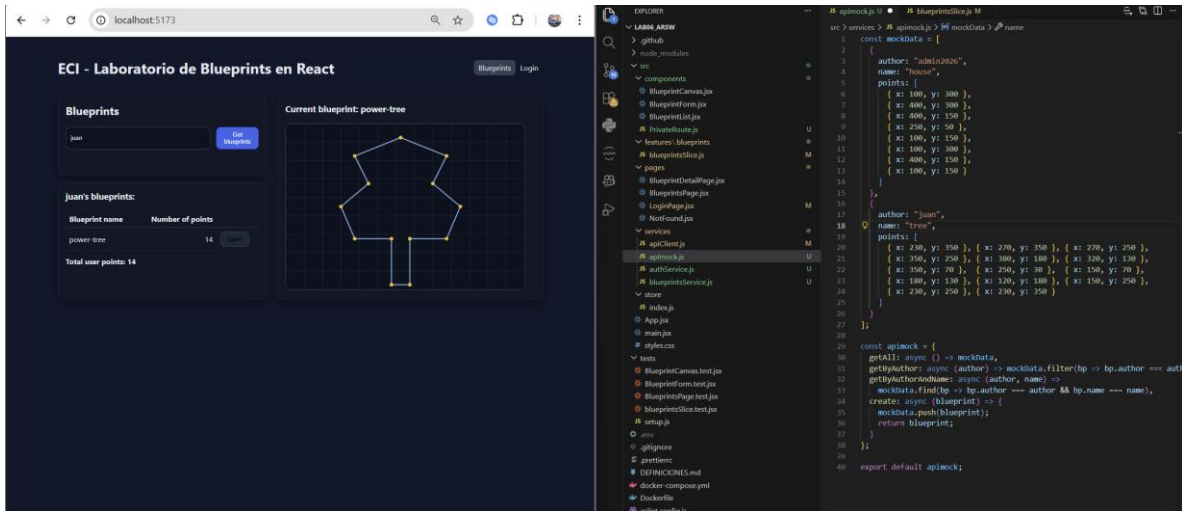
```
src > services > # blueprintsService.js > default
1 import apimock from './apimock';
2 import apiclient from './apiclient';
3
4 const useMock = import.meta.env.VITE_USE MOCK === 'true';
5 const blueprintsService = useMock ? apimock : apiclient;
6
7 export default blueprintsService;
```



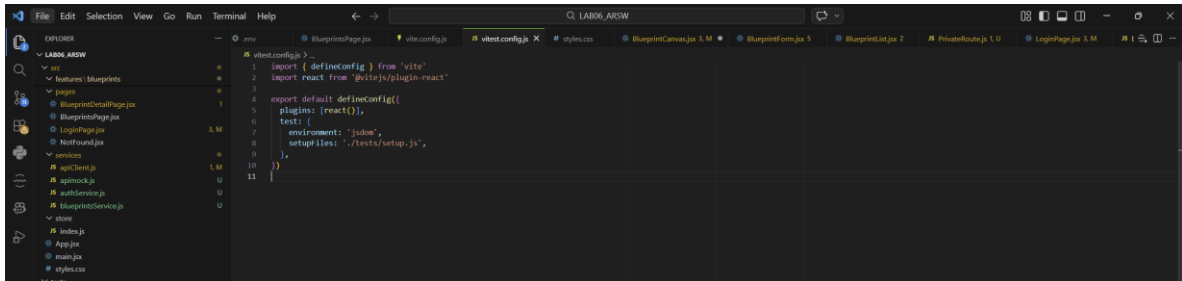
```
src > features > blueprints > # blueprintsSlice.js > # fetchAuthors() @ createAsyncThunk(blueprints.fetchBlueprint) callback
1 import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
2 import blueprintsService from '../../services/blueprintsService.js';
3
4 export const fetchAuthors = createAsyncThunk('blueprints/fetchAuthors', async () => {
5   const data = await blueprintsService.getAll();
6   const authors = [...new Set(data.map(bp => bp.author))];
7   return authors;
8 });
9
10 export const fetchByAuthor = createAsyncThunk('blueprints/fetchByAuthor', async (author) => {
11   const data = await blueprintsService.getbyAuthor(author);
12   return { author, items: data };
13 });
14
15 export const fetchBlueprint = createAsyncThunk(
16   'blueprints/fetchBlueprint',
17   async ({ author, name }) => {
18     const data = await blueprintsService.getbyAuthorAndName(author, name);
19     return data;
20   }
21 );
22
23 export const createBlueprint = createAsyncThunk('blueprints/createBlueprint', async (payload) => {
24   const data = await blueprintsService.create(payload);
25   return data;
26 });
27
28 const slice = createSlice({
29   name: 'blueprints',
30   initialState: {
31     authors: [],
32     byAuthor: [],
33   },
34 });
```



```
src > vite.config.js
1 VITE_API_BASE_URL=http://localhost:8080/api
2 VITE_USE MOCK = 'true'
```



- Para pruebas unitarias



- Navegador “falso” para las pruebas, se engaña con un mock

